

Superstore Sales Analysis

Subqueries, CTEs & Window Functions
SQL Queries, Screenshots & Results

1. Step 1 — Setup Data (superstore_raw, customers, orders, products)
2. 2.1 — Orders where sales > average sales (Subquery)
3. 2.2 — Highest sales order for each customer (Subquery)
4. 2.3 — Total sales for each customer (CTE)
5. 2.4 — Customers whose total sales are above average (CTE + Subquery)
6. 2.5 — Rank all customers based on total sales (Window Function)
7. 2.6 — Row numbers for each order within a customer (Window + PARTITION BY)
8. 2.7 — Top 3 customers based on total sales (Window Function)
9. Step 3 — Final Combined Query (JOIN + CTE + Window Function)
10. Mini Project 1 — Top 5 customers
11. Mini Project 2 — Bottom 5 customers
12. Mini Project 3 — Customers who made only one order
13. Mini Project 4 — Customers with above-average sales
14. Mini Project 5 — Highest order value per customer

Dataset: Sample - Superstore.csv | 793 customers | 1,862 products | 9,994 orders | Total sales: \$2,297,200.86

Step 1

SETUP DATA

Question

Import the Superstore dataset into a table named **superstore_raw**, then create **customers**, **orders**, and **products** tables and populate them using **SELECT DISTINCT**.

SQL Query

```
SELECT 'superstore_raw' AS table_name, COUNT(*) AS row_count FROM superstore_raw
UNION ALL SELECT 'customers', COUNT(*) FROM customers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'orders', COUNT(*) FROM orders;
```

Query Output

Step 1 - Setup Data (table row counts)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

No object selected

Information

Query 1

```
138 SELECT 'superstore_raw' AS table_name, COUNT(*) AS row_count FROM su
139 UNION ALL SELECT 'customers', COUNT(*) FROM customers
140 UNION ALL SELECT 'products', COUNT(*) FROM products
141 UNION ALL SELECT 'orders', COUNT(*) FROM orders;
```

SQL File 1

Result Grid

Messages

Filter rows...

Grid Export/Import View Cell

#	table_name	row_count
1	superstore_raw	9994
2	customers	793
3	products	1862
4	orders	9994

✓ 4 row(s) fetched

Full CREATE TABLE / INSERT ... SELECT DISTINCT statements are in the accompanying .sql script.

2.1

SUBQUERY

Question

Find all orders where sales are greater than the average sales.

SQL Query

```
SELECT row_id, order_id, customer_id, sales
FROM orders
WHERE sales > (SELECT AVG(sales) FROM orders)
ORDER BY sales DESC;
```

Query Output

2.1 - Orders where sales > average sales (Subquery)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

No object selected

Information

Query 1 SQL File 1

```
138 SELECT row_id, order_id, customer_id, sales
139 FROM orders
140 WHERE sales > (SELECT AVG(sales) FROM orders)
141 ORDER BY sales DESC;
```

Result Grid Messages

Filter rows... Grid Export/Import View Cell

#	row_id	order_id	customer_id	sales
1	2698	CA-2014-145317	SM-20320	22638.48
2	6827	CA-2016-118689	TC-20980	17499.95
3	8154	CA-2017-140151	RB-19360	13999.96
4	2624	CA-2017-127180	TA-21385	11199.97
5	4191	CA-2017-166709	HL-15040	10499.97
6	9040	CA-2016-117121	AB-10105	9892.74
7	4099	CA-2014-116904	SC-20095	9449.95
8	4278	US-2016-107440	BS-11365	9099.93
9	8489	CA-2016-158841	SE-20110	8749.95
10	6426	CA-2016-143714	CC-12370	8399.98
11	2506	CA-2014-143917	KL-16645	8187.65
12	166	CA-2014-139892	BM-11140	8159.95

✓ 2360 row(s) fetched (of 9,994 total orders)

2,360 of 9,994 orders (23.6%) have sales above the average of ~\$229.86.

2.2

SUBQUERY

Question

Find the highest sales order for each customer.

SQL Query

```
SELECT o.customer_id, c.customer_name, o.order_id, o.sales
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.sales = (
    SELECT MAX(o2.sales) FROM orders o2
    WHERE o2.customer_id = o.customer_id
)
ORDER BY o.sales DESC;
```

Query Output

2.2 - Highest sales order for each customer (Subquery)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

No object selected

Information

Query 1

```
138 SELECT o.customer_id, c.customer_name, o.order_id, o.sales
139 FROM orders o
140 JOIN customers c ON c.customer_id = o.customer_id
141 WHERE o.sales = (
142     SELECT MAX(o2.sales) FROM orders o2
143     WHERE o2.customer_id = o.customer_id
144 )
145 ORDER BY o.sales DESC;
```

SQL File 1

Result Grid

Filter rows...

Grid Export/Import View Cell

#	customer_id	customer_name	order_id	sales
1	SM-20320	Sean Miller	CA-2014-145317	22638.48
2	TC-20980	Tamara Chand	CA-2016-118689	17499.95
3	RB-19360	Raymond Buch	CA-2017-140151	13999.96
4	TA-21385	Tom Ashbrook	CA-2017-127180	11199.97
5	HL-15040	Hunter Lopez	CA-2017-166709	10499.97
6	AB-10105	Adrian Barton	CA-2016-117121	9892.74
7	SC-20095	Sanjit Chand	CA-2014-116904	9449.95
8	BS-11365	Bill Shonely	US-2016-107440	9099.93
9	SE-20110	Sanjit Engle	CA-2016-158841	8749.95
10	CC-12370	Christopher Conant	CA-2016-143714	8399.98
11	KL-16645	Ken Lonsdale	CA-2014-143917	8187.65
12	BM-11140	Becky Martin	CA-2014-139892	8159.95

✓ 795 row(s) fetched (1 per customer, 793 customers)

Returns one row per customer (793 rows) — each customer's single largest order.

2.3

CTE

Question

Calculate total sales for each customer.

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT * FROM customer_totals  
ORDER BY total_sales DESC;
```

Query Output

2.3 - Total sales for each customer (CTE)

FileEditViewQueryDatabaseServerToolsWindowHelp

Filter objects

superstore_db

Tables

customers

orders

products

superstore_raw

Views

Procedures

Functions

Triggers

Query 1

SQL File 1

138

WITH customer_totals AS (

139

SELECT c.customer_id, c.customer_name,

140

SUM(o.sales) AS total_sales

141

FROM customers c

142

JOIN orders o ON o.customer_id = c.customer_id

143

GROUP BY c.customer_id, c.customer_name

144

)

145

SELECT * FROM customer_totals

146

ORDER BY total_sales DESC;

Result Grid

Messages

Filter rows...

Grid

Export/Import

View Cell

#	customer_id	customer_name	total_sales
1	SM-20320	Sean Miller	25043.05
2	TC-20980	Tamara Chand	19052.22
3	RB-19360	Raymond Buch	15117.34
4	TA-21385	Tom Ashbrook	14595.62
5	AB-10105	Adrian Barton	14473.57
6	KL-16645	Ken Lonsdale	14175.23
7	SC-20095	Sanjit Chand	14142.33
8	HL-15040	Hunter Lopez	12873.3
9	SE-20110	Sanjit Engle	12209.44
10	CC-12370	Christopher Conant	12129.07
11	TS-21370	Todd Sumrall	11891.75
12	GT-14710	Greg Tran	11820.12

No object selected

Information

793 row(s) fetched (793 customers)

793 rows returned — one per customer, ranked highest to lowest total sales.

2.4

CTE + SUBQUERY

Question

Find customers whose total sales are above average.

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT * FROM customer_totals  
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_totals)  
ORDER BY total_sales DESC;
```

Query Output

2.4 - Customers whose total sales are above average (CTE + Subquery)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

Query 1 SQL File 1

```
138 WITH customer_totals AS (  
139     SELECT c.customer_id, c.customer_name,  
140           SUM(o.sales) AS total_sales  
141     FROM customers c  
142     JOIN orders o ON o.customer_id = c.customer_id  
143     GROUP BY c.customer_id, c.customer_name  
144 )  
145 SELECT * FROM customer_totals  
146 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_totals)  
147 ORDER BY total_sales DESC;
```

Result Grid Messages

Filter rows... Grid Export/Import View Cell

#	customer_id	customer_name	total_sales
1	SM-20320	Sean Miller	25043.05
2	TC-20980	Tamara Chand	19052.22
3	RB-19360	Raymond Buch	15117.34
4	TA-21385	Tom Ashbrook	14595.62
5	AB-10105	Adrian Barton	14473.57
6	KL-16645	Ken Lonsdale	14175.23
7	SC-20095	Sanjit Chand	14142.33
8	HL-15040	Hunter Lopez	12873.3
9	SE-20110	Sanjit Engle	12209.44
10	CC-12370	Christopher Conant	12129.07
11	TS-21370	Todd Sumrall	11891.75
12	GT-14710	Greg Tran	11820.12

✓ 294 of 793 customers above average (\$2,896.85)

294 of 793 customers (37.1%) have total sales above the average of \$2,896.85.

2.5

WINDOW FUNCTION

Question

Rank all customers based on total sales.

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_id, customer_name, total_sales,  
       RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
FROM customer_totals;
```

Query Output

The screenshot shows a SQL IDE interface with a dark theme. The title bar reads "2.5 - Rank all customers based on total sales (Window Function)". The menu bar includes File, Edit, View, Query, Database, Server, Tools, Window, and Help. On the left, a sidebar shows the database structure for "superstore_db", including tables (customers, orders, products, superstore_raw), views, procedures, functions, and triggers. The main editor displays the SQL query from the previous block. Below the editor, the "Result Grid" tab is active, showing a table with 5 columns: #, customer_id, customer_name, total_sales, and sales_rank. The table contains 12 rows of data, representing the top customers by total sales. At the bottom, a status bar indicates "793 row(s) fetched (793 customers ranked)".

#	customer_id	customer_name	total_sales	sales_rank
1	SM-20320	Sean Miller	25043.05	1
2	TC-20980	Tamara Chand	19052.22	2
3	RB-19360	Raymond Buch	15117.34	3
4	TA-21385	Tom Ashbrook	14595.62	4
5	AB-10105	Adrian Barton	14473.57	5
6	KL-16645	Ken Lonsdale	14175.23	6
7	SC-20095	Sanjit Chand	14142.33	7
8	HL-15040	Hunter Lopez	12873.3	8
9	SE-20110	Sanjit Engle	12209.44	9
10	CC-12370	Christopher Conant	12129.07	10
11	TS-21370	Todd Sumrall	11891.75	11
12	GT-14710	Greg Tran	11820.12	12

RANK() gives every one of the 793 customers a sales rank; tied totals share a rank.

2.6

WINDOW FUNCTION + PARTITION BY

Question

Assign row numbers to each order within a customer.

SQL Query

```
SELECT row_id, order_id, customer_id, order_date, sales,
       ROW_NUMBER() OVER (
         PARTITION BY customer_id
         ORDER BY order_date, row_id
       ) AS order_seq_num
FROM orders
WHERE customer_id = 'AA-10315' -- example customer
ORDER BY order_seq_num;
```

Query Output

2.6 - Row number per order within a customer (Window + PARTITION BY)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

Query 1

```
138 SELECT row_id, order_id, customer_id, order_date, sales,
139       ROW_NUMBER() OVER (
140         PARTITION BY customer_id
141         ORDER BY order_date, row_id
142       ) AS order_seq_num
143 FROM orders
144 WHERE customer_id = 'AA-10315' -- example customer
145 ORDER BY order_seq_num;
```

Result Grid

Grid Export/Import View Cell

#	row_id	order_id	customer_id	order_date	sales	order_seq_num
1	1300	CA-2015-121391	AA-10315	10/4/2015	26.96	1
2	5199	CA-2016-103982	AA-10315	3/3/2016	3930.07	2
3	5200	CA-2016-103982	AA-10315	3/3/2016	2.3	3
4	5201	CA-2016-103982	AA-10315	3/3/2016	431.98	4
5	5202	CA-2016-103982	AA-10315	3/3/2016	41.72	5
6	2230	CA-2014-128055	AA-10315	3/31/2014	673.57	6
7	2231	CA-2014-128055	AA-10315	3/31/2014	52.98	7
8	1160	CA-2017-147039	AA-10315	6/29/2017	362.94	8
9	1161	CA-2017-147039	AA-10315	6/29/2017	11.54	9
10	7469	CA-2014-138100	AA-10315	9/15/2014	14.94	10
11	7470	CA-2014-138100	AA-10315	9/15/2014	14.56	11

✓ showing customer AA-10315 - full query covers all 9,994 rows

Shown here for one example customer (AA-10315); removing the WHERE clause numbers every order line for all customers across all 9,994 rows.

2.7

WINDOW FUNCTION

Question

Display the top 3 customers based on total sales.

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
ranked AS (  
    SELECT customer_id, customer_name, total_sales,  
           DENSE_RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
    FROM customer_totals  
)  
SELECT * FROM ranked WHERE sales_rank <= 3;
```

Query Output

2.7 - Top 3 customers based on total sales (Window Function)

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

Query 1

```
138 WITH customer_totals AS (  
139     SELECT c.customer_id, c.customer_name,  
140           SUM(o.sales) AS total_sales  
141     FROM customers c  
142     JOIN orders o ON o.customer_id = c.customer_id  
143     GROUP BY c.customer_id, c.customer_name  
144 ),  
145 ranked AS (  
146     SELECT customer_id, customer_name, total_sales,  
147           DENSE_RANK() OVER (ORDER BY total_sales DESC) AS sales_ra  
148     FROM customer_totals  
149 )  
150 SELECT * FROM ranked WHERE sales_rank <= 3;
```

Result Grid

Filter rows... Grid Export/Import View Cell

#	customer_id	customer_name	total_sales	sales_rank
1	SM-20320	Sean Miller	25043.05	1
2	TC-20980	Tamara Chand	19052.22	2
3	RB-19360	Raymond Buch	15117.34	3

✓ 3 row(s) fetched

Sean Miller, Tamara Chand, and Raymond Buch are the top 3 customers by total sales.

Step 3

JOIN + CTE + WINDOW FUNCTION

Question

Write one final query that shows Customer Name, Total Sales, and Rank, combining a JOIN, a CTE, and a window function.

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT ct.customer_name, ct.total_sales,  
       RANK() OVER (ORDER BY ct.total_sales DESC) AS sales_rank  
FROM customer_totals ct  
ORDER BY sales_rank;
```

Query Output

The screenshot displays a SQL IDE window titled "Step 3 - Final Combined Query (JOIN + CTE + Window Function)". The interface includes a menu bar (File, Edit, View, Query, Database, Server, Tools, Window, Help), a toolbar with color-coded buttons, and a sidebar for "Filter objects" showing a database structure for "superstore_db" with tables like customers, orders, products, and superstore_raw. The main editor shows the SQL query from the previous block, with line numbers 138 to 148. Below the editor, the "Result Grid" tab is active, showing a table with 12 rows of data. The table has columns: #, customer_name, total_sales, and sales_rank. The data is sorted by total_sales in descending order. At the bottom, a status bar indicates "793 row(s) fetched (793 customers)".

#	customer_name	total_sales	sales_rank
1	Sean Miller	25043.05	1
2	Tamara Chand	19052.22	2
3	Raymond Buch	15117.34	3
4	Tom Ashbrook	14595.62	4
5	Adrian Barton	14473.57	5
6	Ken Lonsdale	14175.23	6
7	Sanjit Chand	14142.33	7
8	Hunter Lopez	12873.3	8
9	Sanjit Engle	12209.44	9
10	Christopher Conant	12129.07	10
11	Todd Sumrall	11891.75	11
12	Greg Tran	11820.12	12

The single combined deliverable requested in Step 3 — all 793 customers ranked by total sales.

Mini Project 1

BUSINESS QUESTION

Question

Who are the top 5 customers?

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_name, total_sales FROM customer_totals  
ORDER BY total_sales DESC LIMIT 5;
```

Query Output

Mini Project 1 - Top 5 customers

File Edit View Query Database Server Tools Window Help

Filter objects

superstore_db

Tables

customers

orders

products

superstore_raw

Views

Procedures

Functions

Triggers

No object selected

Information

Query 1

SQL File 1

138 WITH customer_totals AS (
139 SELECT c.customer_id, c.customer_name,
140 SUM(o.sales) AS total_sales
141 FROM customers c
142 JOIN orders o ON o.customer_id = c.customer_id
143 GROUP BY c.customer_id, c.customer_name
144)
145 SELECT customer_name, total_sales FROM customer_totals
146 ORDER BY total_sales DESC LIMIT 5;

Result Grid

Messages

Filter rows... Grid Export/Import View Cell

#	customer_name	total_sales
1	Sean Miller	25043.05
2	Tamara Chand	19052.22
3	Raymond Buch	15117.34
4	Tom Ashbrook	14595.62
5	Adrian Barton	14473.57

✓ 5 row(s) fetched

Top 5 customers together account for \$88,281.80 in sales (3.8% of total revenue).

Mini Project 2

BUSINESS QUESTION

Question

Who are the bottom 5 customers?

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_name, total_sales FROM customer_totals  
ORDER BY total_sales ASC LIMIT 5;
```

Query Output

Mini Project 2 - Bottom 5 customers

File Edit View Query Database Server Tools Window Help

Filter objects

superstore_db

- Tables
 - customers
 - orders
 - products
 - superstore_raw
- Views
- Procedures
- Functions
- Triggers

No object selected
Information

Query 1

SQL File 1

138 WITH customer_totals AS (
139 SELECT c.customer_id, c.customer_name,
140 SUM(o.sales) AS total_sales
141 FROM customers c
142 JOIN orders o ON o.customer_id = c.customer_id
143 GROUP BY c.customer_id, c.customer_name
144)
145 SELECT customer_name, total_sales FROM customer_totals
146 ORDER BY total_sales ASC LIMIT 5;

Result Grid

Messages

Filter rows... Grid Export/Import View Cell

#	customer_name	total_sales
1	Thais Sissman	4.83
2	Lela Donovan	5.3
3	Carl Jackson	16.52
4	Mitch Gastineau	16.74
5	Roy Skaria	22.33

✓ 5 row(s) fetched

The bottom 5 customers all have total sales under \$25 — likely single small purchases.

Mini Project 3

BUSINESS QUESTION

Question

Which customers made only one order?

SQL Query

```
SELECT c.customer_id, c.customer_name,  
       COUNT(DISTINCT o.order_id) AS order_count  
FROM customers c  
JOIN orders o ON o.customer_id = c.customer_id  
GROUP BY c.customer_id, c.customer_name  
HAVING COUNT(DISTINCT o.order_id) = 1  
ORDER BY c.customer_name;
```

Query Output

The screenshot shows a SQL IDE window titled "Mini Project 3 - Customers who made only one order". The interface includes a menu bar (File, Edit, View, Query, Database, Server, Tools, Window, Help), a toolbar with colored buttons, and a sidebar for "Filter objects" showing a database structure for "superstore_db" with tables like customers, orders, products, and superstore_raw. The main editor displays the SQL query from the previous block. Below the editor, the "Result Grid" tab is active, showing a table with 12 rows of results. The table has columns: #, customer_id, customer_name, and order_count. The status bar at the bottom indicates "12 row(s) fetched".

#	customer_id	customer_name	order_count
1	AR-10570	Anemone Ratner	1
2	AO-10810	Anthony O'Donnell	1
3	CJ-11875	Carl Jackson	1
4	JC-15385	Jenna Caffey	1
5	JR-15700	Jocasta Rupert	1
6	LD-16855	Lela Donovan	1
7	MG-18205	Mitch Gastineau	1
8	PH-18790	Patricia Hirasaki	1
9	RE-19405	Ricardo Emerson	1
10	RM-19750	Roland Murray	1
11	SM-20905	Susan MacKendrick	1
12	TC-21145	Theresa Coyne	1

12 customers have placed exactly one order — good candidates for a win-back campaign.

Mini Project 4

BUSINESS QUESTION

Question

Which customers have above-average sales?

SQL Query

```
WITH customer_totals AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales  
    FROM customers c  
    JOIN orders o ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_name, total_sales FROM customer_totals  
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_totals)  
ORDER BY total_sales DESC;
```

Query Output

Mini Project 4 - Customers with above-average sales

File Edit View Query Database Server Tools Window Help

Filter objects

superstore_db

- Tables
 - customers
 - orders
 - products
 - superstore_raw
- Views
- Procedures
- Functions
- Triggers

No object selected

Information

Query 1

```
138 WITH customer_totals AS (  
139     SELECT c.customer_id, c.customer_name,  
140            SUM(o.sales) AS total_sales  
141     FROM customers c  
142     JOIN orders o ON o.customer_id = c.customer_id  
143     GROUP BY c.customer_id, c.customer_name  
144 )  
145 SELECT customer_name, total_sales FROM customer_totals  
146 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_totals)  
147 ORDER BY total_sales DESC;
```

Result Grid

Filter rows... Grid Export/Import View Cell

#	customer_name	total_sales
1	Sean Miller	25043.05
2	Tamara Chand	19052.22
3	Raymond Buch	15117.34
4	Tom Ashbrook	14595.62
5	Adrian Barton	14473.57
6	Ken Lonsdale	14175.23
7	Sanjit Chand	14142.33
8	Hunter Lopez	12873.3
9	Sanjit Engle	12209.44
10	Christopher Conant	12129.07
11	Todd Sumrall	11891.75
12	Greg Tran	11820.12

✓ 294 of 793 customers fetched (37.1% above average)

294 of 793 customers (37.1%) sit above the \$2,896.85 average total sales.

Mini Project 5

BUSINESS QUESTION

Question

What is the highest order value per customer?

SQL Query

```
SELECT c.customer_name, MAX(o.sales) AS highest_order_value
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY highest_order_value DESC;
```

Query Output

Mini Project 5 - Highest order value per customer

File Edit View Query Database Server Tools Window Help

Filter objects

- superstore_db
 - Tables
 - customers
 - orders
 - products
 - superstore_raw
 - Views
 - Procedures
 - Functions
 - Triggers

No object selected

Information

Query 1

```
138 SELECT c.customer_name, MAX(o.sales) AS highest_order_value
139 FROM customers c
140 JOIN orders o ON o.customer_id = c.customer_id
141 GROUP BY c.customer_name
142 ORDER BY highest_order_value DESC;
```

SQL File 1

Result Grid

Messages

Filter rows... Grid Export/Import View Cell

#	customer_name	highest_order_value
1	Sean Miller	22638.48
2	Tamara Chand	17499.95
3	Raymond Buch	13999.96
4	Tom Ashbrook	11199.97
5	Hunter Lopez	10499.97
6	Adrian Barton	9892.74
7	Sanjit Chand	9449.95
8	Bill Shonely	9099.93
9	Sanjit Engle	8749.95
10	Christopher Conant	8399.98
11	Ken Lonsdale	8187.65
12	Becky Martin	8159.95

✓ 793 row(s) fetched (793 customers)

Sean Miller's single largest order (\$22,638.48) is the highest order value in the entire dataset.